CI/CD avec GitHub Actions : mes notes

Programme DevOps — Jour 3, étapes 1 à 3

Hicham Habchane

27 octobre 2026

Dépôt : `https://github.com/H-HABCHANE/DevOps-learning-labs`

Table des matières

1 L'idée de la CI/CD **3**

2 L'appli Python, testée en local **3**

2.1 Structure . . . 3
2.2 Le code et les tests . . . 4
2.3 Versions épinglées . . . 4
2.4 Codes de sortie . . . 4
2.5 Lire un échec pytest . . . 4
2.6 Lint avec ruff . . . 5

3 Git et GitHub **5**

3.1 Ce qu'on n'envoie pas . . . 5
3.2 Les deux temps de git . . . 5
3.3 Créer le dépôt GitHub . . . 5

4 Le premier workflow **6**

5 Pull requests et protection de main **7**

5.1 Le cycle de travail . . . 7
5.2 Ruleset `protection-main` . . . 7
5.3 Lire une PR et l'historique . . . 7

6 Mémo des commandes **8**

6.1 Shell . . . 8
6.2 Python . . . 8
6.3 Git . . . 8

7 Questions et réponses **9**

7.1 Vocabulaire 9
7.2 Appli Python, tests et lint 9
7.3 Git et GitHub 9
7.4 Workflow et pull requests 10

8 La suite **10**

1. L'idée de la CI/CD

Le problème : à la main, on lance les tests, on construit l'image Docker, on la pousse. Un oubli, et on publie quelque chose de cassé.

— **CI** (*Continuous Integration*, intégration continue) : à chaque modification, le code est vérifié automatiquement (tests, lint).
— **CD** (*Continuous Delivery / Deployment*, livraison / déploiement continu) : si les vérifications passent, on publie (image Docker) ou on déploie automatiquement.

L'analogie du restaurant : à chaque modification de recette, un contrôleur qualité goûte le plat avant qu'il parte en salle.

GitHub Actions	Restaurant	En vrai
Workflow	la fiche de procédure qualité	fichier YAML dans `.github/workflows/`
Event	« une recette a été modifiée »	`push`, `pull_request`, horaire, manuel
Job	un poste de contrôle	groupe de steps sur **une** machine
Step	une action précise du poste	`run: pytest` ou `uses: actions/checkout@v5`
Runner	le contrôleur qui travaille	VM neuve, ex. `ubuntu-latest`

Trois règles

1. Le runner est une machine **neuve et vide** à chaque exécution : il faut récupérer le code (`checkout`) et installer les dépendances.
2. Les steps d'un job s'exécutent **dans l'ordre** sur la même machine (les fichiers restent entre steps). Une step qui échoue (code de sortie $\neq 0$) arrête le job, qui devient rouge.
3. Les jobs tournent **en parallèle**, chacun sur sa machine, sans rien partager. `needs:` impose un ordre et une condition de réussite.

```
jobs:
  tests:
    ...
  publier-image:
    needs: tests        # attend que "tests" soit vert, sinon il est saute
    ...
```

2. L'appli Python, testée en local

Principe : la CI refait automatiquement ce qu'on sait faire à la main. On valide donc tout en local d'abord.

2.1 Structure

```
02-appli-python/
|-- app/calcul.py           # prix_ttc(prix_ht, tva=0.10)
|-- tests/test_calcul.py    # 3 tests pytest
`-- requirements-dev.txt    # outils de verification, versions epinglees
```

2.2 Le code et les tests

```
def prix_ttc(prix_ht: float, tva: float = 0.10) -> float:
    """Prix TTC arrondi au centime (TVA restauration : 10 %)."""
    if prix_ht < 0:
        raise ValueError("le prix ne peut pas être négatif")
    return round(prix_ht * (1 + tva), 2)
```

```
import pytest

from app.calcul import prix_ttc


def test_tva_par_defaut():
    assert prix_ttc(10) == 11.0


def test_tva_personnalisee():
    assert prix_ttc(10, 0.20) == 12.0


def test_prix_negatif_refuse():
    with pytest.raises(ValueError):
        prix_ttc(-5)
```

— pytest trouve tout seul les fichiers `test_*.py` et les fonctions `test_*`.
— `assert X == Y` : si c'est faux, le test échoue.
— `with pytest.raises(ValueError):` le bloc *doit* lever cette erreur ; on teste aussi les mauvais cas.

2.3 Versions épinglées

```
# requirements-dev.txt
pytest==9.1.1
ruff==0.17.3
```

Mêmes versions en local et sur le runner : build **reproductible**.

2.4 Codes de sortie

GitHub Actions ne lit qu'une chose : le **code de sortie**. `0` : step réussie. Autre chose : step en échec, job rouge, steps suivantes sautées.

Code pytest	Sens
0	tous les tests passent
1	au moins un test échoue
5	aucun test trouvé (évite un faux vert)

2.5 Lire un échec pytest

```
tests/test_calcul.py F..                    # une lettre par test : F = echec, . = OK
>       assert prix_ttc(10) == 11.5         # > : la ligne qui casse
E       assert 11.0 == 11.5                 # E : obtenu vs attendu
tests/test_calcul.py:7: AssertionError      # fichier:ligne
```

À retenir : un test rouge ne veut pas dire que le code est faux : le test lui-même peut être erroné.

2.6 Lint avec ruff

pytest vérifie que le code **fait la bonne chose** ; ruff vérifie qu'il est **propre**, sans l'exécuter.

Code	Sens	Famille
`F401`	import inutilisé	Pyflakes (erreurs logiques probables)
`I001`	imports mal organisés	isort (ordre et forme des imports)

`[*]` = corrigeable automatiquement par `--fix`.

À retenir : en local `ruff check --fix .` pour corriger ; en CI `ruff check .` pour vérifier. La CI doit signaler un problème, pas le cacher.

3. Git et GitHub

3.1 Ce qu'on n'envoie pas

```
# .gitignore
.venv/
__pycache__/
.pytest_cache/
.ruff_cache/
```

On envoie la recette, pas la vaisselle sale. Le `.venv` (52 Mo, lié à la machine) se reconstruit depuis `requirements-dev.txt`. `.git/info/exclude` : liste d'exclusion locale, jamais publiée.

3.2 Les deux temps de git

`git add` pose les plats sur le passe-plat (zone de préparation) ; `git commit` envoie le service (photo dans l'historique). **Toujours relire `git status` avant de commiter.**

— `git add <fichier>` : précis, à préférer.
— `git commit -am` : embarque *tous* les fichiers suivis modifiés, même ceux oubliés. Aveugle.

À retenir : un secret commité reste dans l'historique même s'il est supprimé ensuite. S'il a été poussé : le considérer compromis et le révoquer immédiatement.

3.3 Créer le dépôt GitHub

Sur `https://github.com/new` : ne **rien** cocher (README, licence…), sinon GitHub crée un commit sans ancêtre commun avec le dépôt local et le push est refusé.

4. Le premier workflow

```yaml
name: CI

on:
  push:
    branches: [main]       # push : seulement sur main
  pull_request:            # toute PR (opened, synchronize, reopened)

permissions:
  contents: read           # moindre privilege : lecture seule

jobs:
  tests:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: 02-appli-python
    steps:
      - name: Recuperer le code
        uses: actions/checkout@v5

      - name: Installer Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.14"
          cache: pip
          cache-dependency-path: 02-appli-python/requirements-dev.txt

      - name: Installer les outils
        run: pip install -r requirements-dev.txt

      - name: Lint (ruff)
        run: ruff check .

      - name: Tests (pytest)
        run: python -m pytest -v
```

Bloc	Sens
`.github/workflows/`	seul emplacement ; GitHub lance tout `.yml` placé ici
`on: push: branches: [main]`	déclenché par un push sur `main` uniquement
`on: pull_request:`	déclenché par une PR : on vérifie *avant* de fusionner
`permissions: contents: read`	le workflow ne peut que lire le dépôt
`working-directory`	chaque `run:` s'exécute dans `02-appli-python`
`uses: ...@v5`	action toute faite, version épinglée
`cache: pip`	cache des téléchargements ; clé = empreinte de `requirements-dev.txt`

— **Fail fast :** ruff (1 s) avant pytest. Si ruff échoue, pytest est sauté.
— Premier run : vert en 25 s, 3 PASSED. Cache : `Cache not found` au premier run, sauvegarde dans la step *Post*, puis `Cache restored`.
— YAML : l'indentation compte (2 espaces, jamais de tabulation).

5. Pull requests et protection de main

5.1 Le cycle de travail

```
branche -> commit -> push -> PR -> CI rouge -> correction -> CI verte -> merge -> CI sur
    ↪ main
                                        ^
                ruleset : merge interdit tant que ce n'est pas vert
```

Action	Event	Run ?
Push sur une branche `test-pr`	`push`	non (filtre `[main]`)
Ouverture de la PR	`pull_request` (`opened`)	oui
Nouveau commit poussé sur la branche de la PR	`pull_request` (`synchronize`)	oui
Merge de la PR	`push` sur `main`	oui

5.2 Ruleset `protection-main`

Settings → *Rules* → *Rulesets* → *New branch ruleset* : cible = branche par défaut, règle *Require status checks to pass* avec le check `tests`.

— Sans ruleset, un check rouge est **informatif** : le merge reste possible.
— Avec, le merge est bloqué tant que la CI n'est pas verte, et le push direct sur `main` est refusé.

À retenir : la CI détecte le problème ; c'est la protection de branche qui l'empêche d'atteindre `main`.

5.3 Lire une PR et l'historique

— Une PR montre la différence **finale** entre les branches ; l'onglet Commits montre le chemin. Un fichier cassé puis réparé n'apparaît pas dans le diff.
— La CI teste toujours le **dernier** commit de la branche.
— `Your branch is up to date with 'origin/main'` compare à la dernière copie connue ; seul `fetch`/`pull` interroge GitHub.

```
*   c263f42 Merge pull request #1 from H-HABCHANE/test-pr
|\
| * 2a938d2 Correction du test : 10 EUR HT = 11 EUR TTC
| * fd57b4b Test volontairement casse pour tester la CI sur PR
|/
* 0c20cc2 CI : workflow GitHub Actions (ruff + pytest)
* 653b52d Jour 3 : vocabulaire GitHub Actions et appli Python testee
```

6. Mémo des commandes

6.1 Shell

```
mkdir -p 02-appli-python/app          # creer un dossier et ses parents
cmd1 && cmd2                          # cmd2 seulement si cmd1 reussit (code 0)
cmd1 ; cmd2                           # cmd2 dans tous les cas
echo "Code de sortie : $?"            # code de sortie de la derniere commande
printf 'a\nb\n' > fichier             # ecrire ; > ECRASE (vide d'abord le fichier)
printf 'a\n' >> fichier               # ajouter a la fin
cat > fichier <<'EOF'                 # heredoc : plusieurs lignes jusqu'a EOF
...
EOF
sed -i 's/ancien/nouveau/' fichier    # remplacer dans le fichier (-i = in place)
sed -i '1i texte' fichier             # inserer une ligne avant la ligne 1
mv "nom avec espaces.png" ~/Pictures/ # deplacer ; guillemets si espaces
cd ~/CI\&CD                           # \& : le & fait partie du nom
```

6.2 Python

```
python3 -m venv .venv                           # environnement isole
.venv/bin/pip install -r requirements-dev.txt   # outils aux versions exactes
python -m pytest -v                             # tests ; -m ajoute le dossier courant
    ↪ aux imports
python -m pytest -q                             # sortie courte
ruff check .                                    # verifier (CI)
ruff check --fix .                              # corriger (local)
```

6.3 Git

```
git init -b main                      # creer le depot, branche principale main
git status --short                    # ?? non suivi, A ajoute, M modifie
git add <fichier>                     # preparer un fichier precis
git commit -m "message"               # enregistrer
git log --oneline --graph -6          # historique dessine
git show --stat --oneline HEAD        # fichiers du dernier commit
git remote add origin git@github.com:H-HABCHANE/DevOps-learning-labs.git
git push -u origin main               # premier push, memorise la branche suivie
git switch -c ma-branche              # creer une branche et s'y placer
git switch main && git pull           # main a jour avant chaque nouvelle branche
git branch -d ma-branche              # supprimer la branche locale
git push origin --delete ma-branche   # supprimer la branche sur GitHub
```

7. Questions et réponses

7.1 Vocabulaire

Q. Différence entre un job et une step ?

→ Un job est un groupe de steps exécuté sur un seul runner. Une step est une seule commande (`run`) ou action (`uses`). Les steps d'un job partagent la même machine ; deux jobs sont sur deux machines différentes.

Q. Que se passe-t-il avec seulement `run: pytest`, sans checkout ?

→ Échec : le runner est neuf et ne contient pas le code. pytest n'est pas installé ; même installé, il ne trouverait aucun test (code 5).

Q. Comment publier l'image seulement si les tests passent ?

→ `needs: tests` dans le job `publier-image` : il attend la réussite de `tests`, sinon il est sauté.

7.2 Appli Python, tests et lint

Q. Faut-il refaire `pip install` à chaque exécution du workflow ?

→ Oui, le runner est neuf. Le cache (`setup-python` avec `cache: pip`) accélère le téléchargement, mais l'installation se refait.

Q. Pourquoi épingler les versions ?

→ Build reproductible : même code + mêmes versions = même résultat en local et en CI. Sinon une nouvelle version d'un outil peut casser le pipeline sans changement de code.

Q. Que renvoient `prix_ttc(10)` et `prix_ttc(10, 0.20)` ?

→ `11.0` et `12.0`.

Q. Code de sortie de pytest quand un test échoue ? Effet en CI ?

→ `1` : la step échoue, le job devient rouge.

Q. Quand un test échoue, qui a tort ?

→ Pas forcément le code : le test peut être faux (ici `11.5` attendu à tort).

Q. `no tests ran` en CI : vert ou rouge ?

→ Rouge (code 5). Voulu : pas de tests = aucune preuve, on évite un faux vert.

Q. Code de sortie de `ruff check .` s'il trouve une erreur ? Effet ?

→ `1` : la step échoue, les suivantes sont sautées, le job est rouge.

Q. En CI : `ruff check --fix .` ou `ruff check .` ?

→ `ruff check .` : la correction faite sur le runner serait perdue, et la CI doit signaler, pas masquer. On corrige en local avec `--fix`.

Q. Pourquoi ruff a-t-il corrigé 1 erreur alors qu'il en annonçait 2 ?

→ Supprimer `import os` (F401) a fait disparaître le bloc d'imports, donc I001 aussi.

7.3 Git et GitHub

Q. Pourquoi ne pas pousser `.venv/` ?

→ Lié au système et aux chemins de la machine, lourd (52 Mo), non portable. Il se reconstruit depuis `requirements-dev.txt`.

Q. Que montre `git status --short` après `git init` ?
→ `.gitignore`, `PROGRESSION.md`, `01-vocabulaire/`, `02-appli-python/` ; pas `.venv/` ni `CLAUDE.md`. Un dossier entièrement nouveau est résumé en une ligne (`-uall` pour le détail).

Q. Pourquoi relire `git status` avant un commit ?
→ Un secret commité reste dans l'historique ; poussé sur un dépôt public, il est exposé. Il faut alors le révoquer immédiatement.

Q. Pourquoi ne rien cocher à la création du dépôt GitHub ?
→ GitHub créerait un commit sans ancêtre commun avec le dépôt local : le push serait refusé.

Q. Pourquoi `git pull` avant de créer une branche ?
→ Sinon la branche part d'un `main` périmé : code ancien et conflits au moment de la PR.

7.4 Workflow et pull requests

Q. Pourquoi ruff avant pytest ?
→ Fail fast : vérifications rapides d'abord, retour plus rapide ; si ruff échoue, pytest est sauté.

Q. Le workflow se lance-t-il avant le push ?
→ Non : GitHub ne voit que ce qui a été poussé.

Q. Le premier run : quel résultat, quelle durée ?
→ Vert en environ 25 s, 3 PASSED.

Q. Pousser une branche `test-pr` déclenche-t-il le workflow ?
→ Non : `push` est filtré sur `main`. Le run démarre à l'ouverture de la PR. Le filtre évite deux runs identiques par commit de PR.

Q. Que fait la PR avec un test cassé ? Le merge est-il bloqué ?
→ Check rouge (`assert 11.0 == 11.5`), mais le merge reste possible sans protection de branche.

Q. Avec le ruleset, comment fusionner la PR ?
→ Corriger sur la même branche et pousser : la PR suit sa branche, la CI se relance, verte → merge débloqué.

Q. Pousser la correction sur `test-pr` déclenche-t-il un run ?
→ Oui : `pull_request`, type `synchronize`.

Q. Après le merge, quel run apparaît et pourquoi, sans `git push` ?
→ `push` sur `main` : le merge crée un commit sur `main`, ce qui est un push sur `main`.

8. La suite

— Étape 4 : construire l'image Docker et la pousser sur Docker Hub, seulement depuis `main` (livraison continue).
— Étape 5 : secrets GitHub, permissions minimales, gestion des échecs.
— Étape 6 : réparer un pipeline cassé, puis défi du Jour 3 sans aide.
— Jour 8 : déploiement continu sur Kubernetes.